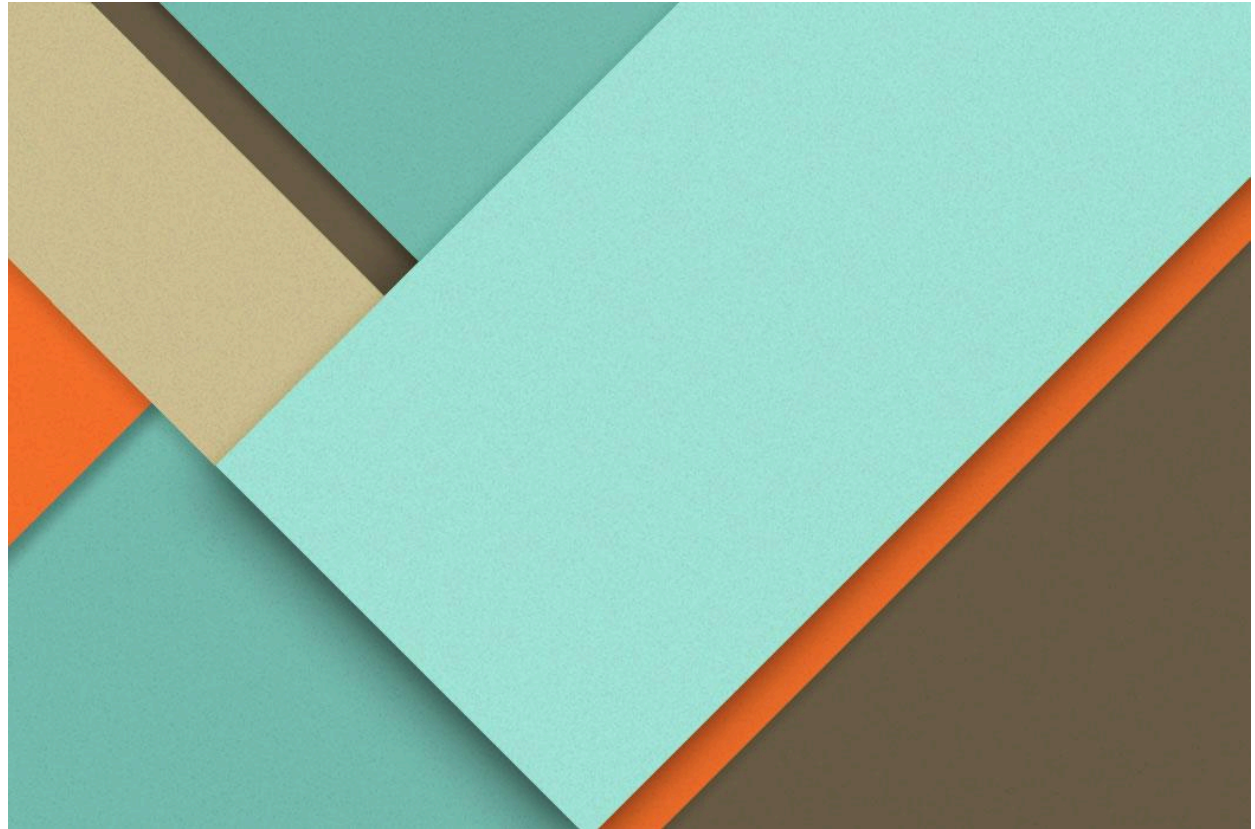




ParcelResolve System Analysis

26.04.2026

Jia Tien
Austin

Chaang Wai Chiu

Amir Hakeem
Amirun Nasrudin

1.0) Overview

This document provides a detailed analysis of an automated logistics assistant designed to streamline the reporting of damaged parcels. The system utilizes a WhatsApp interface to capture user data, which is then processed by a multimodal Artificial Intelligence (AI) model to extract critical shipping information and assess physical damage levels.

2.0) Problem Statement

Current logistics reporting often relies on manual data entry or separate apps that drivers must learn to use. This leads to several issues:

- **Manual Errors:** Tracking numbers are often misread or mistyped during high-volume shifts.
- **Subjective Assessments:** Without a standard tool, "damage" is defined differently by every worker.
- **Input Friction:** Requiring users to navigate complex forms can delay the reporting of damaged goods.

3.0) System Requirements

3.1 Functional Requirements

Functional requirements define the core tasks the system must execute.

- **Message Handling:** The system must listen for incoming POST requests from the Twilio WhatsApp API on port 8080.
- **Content Extraction:** The system must extract the message body (text) and the media URL (image) from the incoming WhatsApp data.
- **Input Validation:** The system must verify the presence of an image; if no image is found, it must block AI processing and request a photo from the user.
- **Multimodal Analysis:** The system must send both text and image data to the Google Gemini-3-flash-preview model for analysis.
- **Automated Data Labeling:** The system must automatically categorize damage into one of four levels: HIGH, MEDIUM, LOW, or NONE.
- **Structured Reporting:** The system must return a valid JSON object containing the `trackingNumber` and `damageLevel`.

3.2 Non-Functional Requirements

Non-functional requirements define the performance and quality standards of the system.

- **Availability:** The system uses a Javalin web server to maintain an active listener for real-time communication.
- **Responsiveness:** The system is configured with a 30-second timeout for both connection and reading to ensure timely responses from the AI agent.
- **Data Integrity:** The system utilizes a strict JSON output format to ensure compatibility with external logistics databases.
- **User Feedback:** The system provides immediate, human-readable feedback to the user via the WhatsApp interface.

4.0) System Architecture

4.1 Architecture Components

- **External Gateway:** Twilio WhatsApp Sandbox handles the messaging interface and routes incoming data to the webhook.
- **Web Server:** A Java-based Javalin application serves as the central hub for receiving and responding to messages.
- **Processing Agent:** The `ExtractionAgent` class manages the heavy lifting, including image-to-Base64 conversion and API communication.
- **Intelligence Layer:** Google Gemini API performs the actual vision-based extraction and logic.

5.0) Data Flow Analysis

5.1 Operational Sequence

1. **User Initiation:** A user sends a WhatsApp message with a parcel photo.
2. **Webhook Trigger:** Twilio sends the data to the `/whatsapp/webhook` endpoint.
3. **Logic Gate:** The system evaluates the `MediaUrl0` parameter.
4. **Data Preparation:** If an image is present, the system converts it into a Base64 string for the AI.
5. **AI Inference:** The system calls the Gemini API with a specific system prompt and the user's data.
6. **Response Generation:** The AI's JSON output is wrapped in TwiML (XML) and sent back to the user's WhatsApp.

6.0) Technical Constraints

- **Format Dependency:** The AI is instructed to output ONLY raw, valid JSON; markdown tags are strictly prohibited to prevent parsing errors.
- **Media Handling:** The system currently defaults to processing images in the `image/jpeg` format.
- **Connectivity:** The system relies on a stable connection to the `generativelanguage.googleapis.com` endpoint to function.